Day 1 · ช่วงเช้า

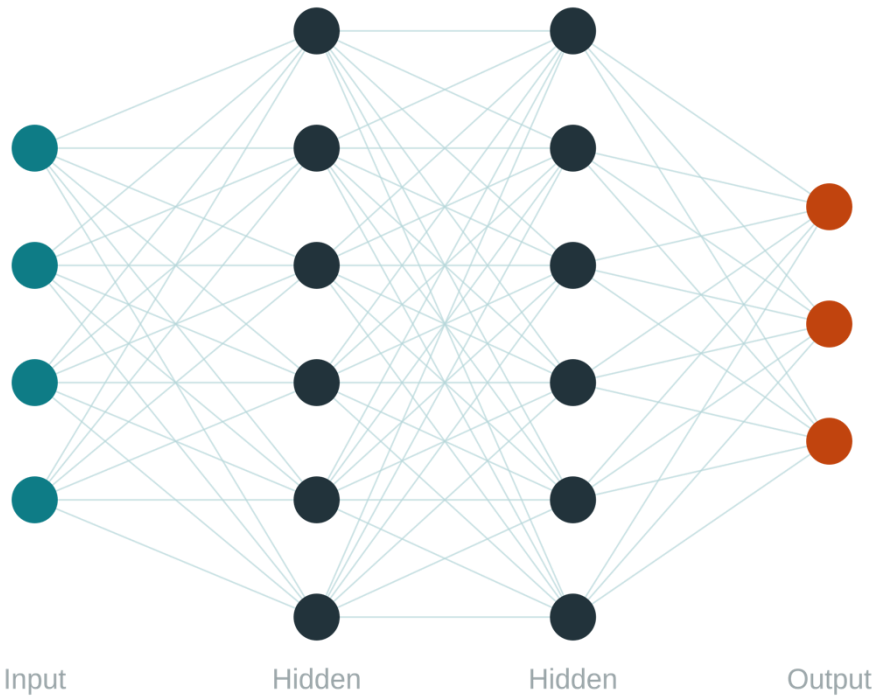
พื้นฐาน Deep Learning

ตั้งแต่ neural network หนึ่งหน่วย จนถึง CNN และ transfer learning

- Neural Network
- CNN
- Transfer Learning

เส้นทางของช่วงเช้า

- 1. หนึ่งหน่วยคำนวณ — perceptron ทำอะไร
- 2. หลายหน่วยหลายชั้น — neural network เรียนรู้อย่างไร
- 3. CNN — เมื่อข้อมูลเป็นภาพ
- 4. Transfer learning — ใช้โมเดลที่ผู้อื่นเทรนไว้แล้ว
- ช่วงบ่ายจะลงมือเขียนโค้ดทั้งสี่ขั้นนี้ด้วย Keras



ส่วนที่ 1

ปูพื้นก่อนเริ่ม

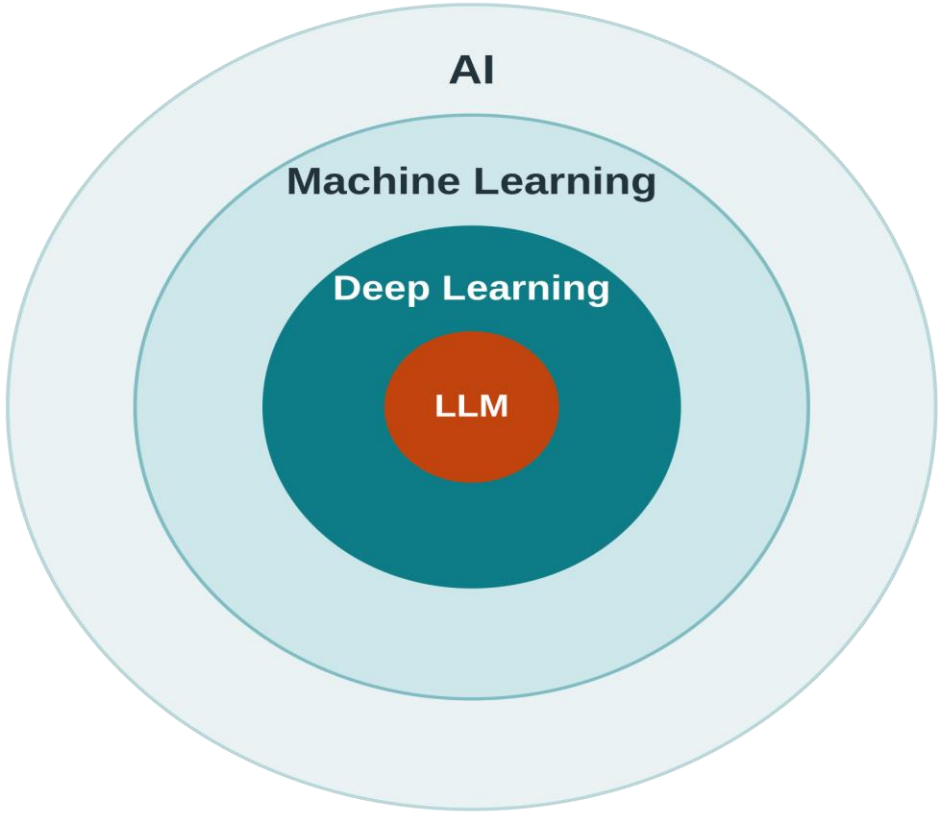
คำศัพท์และแนวคิดที่จะใช้ตลอดทั้งวัน

~30 นาที



ปูพื้น

AI, Machine Learning และ Deep Learning สัมพันธ์กันอย่างไร



สามชั้นซ้อนกัน

- AI — เป้าหมายกว้างที่สุด
- ML — ให้ระบบหารูปแบบจากข้อมูลเอง
- DL — ML ที่ใช้ neural network หลายชั้น
- LLM — จัดอยู่ในกลุ่ม deep learning

ปูพื้น

Machine Learning ต่างจากการเขียนโปรแกรมแบบเดิมอย่างไร

การเขียนโปรแกรมแบบเดิม

- มนุษย์เป็นผู้กำหนดกฎเกณฑ์
- ป้อนข้อมูล + กฎ ได้คำตอบออกมา
- เช่น ถ้าอีเมลมีคำว่า "ถูกรางวัล" = สแปม
- ปัญหา: กฎมีข้อยกเว้นไม่รู้จบ

Machine Learning

- มนุษย์เป็นผู้ป้อนตัวอย่างพร้อมคำตอบ
- ป้อนข้อมูล + คำตอบ ได้กฎเกณฑ์ออกมา
- เช่น ดูอีเมลสแปม/ไม่สแปมอย่างละหมื่นฉบับ
- ระบบสรุปรูปแบบเองว่าอะไรคือสแปม

ปูพื้น

Supervised กับ Unsupervised Learning

Supervised learning

- ข้อมูลฝึกมีคำตอบกำกับมาด้วย
- เรียนจากการเทียบคำตอบทายกับจริง
- เช่น รูปนี้คือแมว รูปนี้คือสุนัข
- เนื้อหาทั้งวันนี้อยู่ในกลุ่มนี้

Unsupervised learning

- ข้อมูลฝึกไม่มีคำตอบกำกับ
- ระบบหาโครงสร้างหรือกลุ่มเอง
- เช่น จัดกลุ่มลูกค้าตามพฤติกรรม
- ไม่ใช่เนื้อหาหลักของคอร์ส

ปูพื้น

โจทย์สองประเภทที่จะพบบ่อยที่สุด

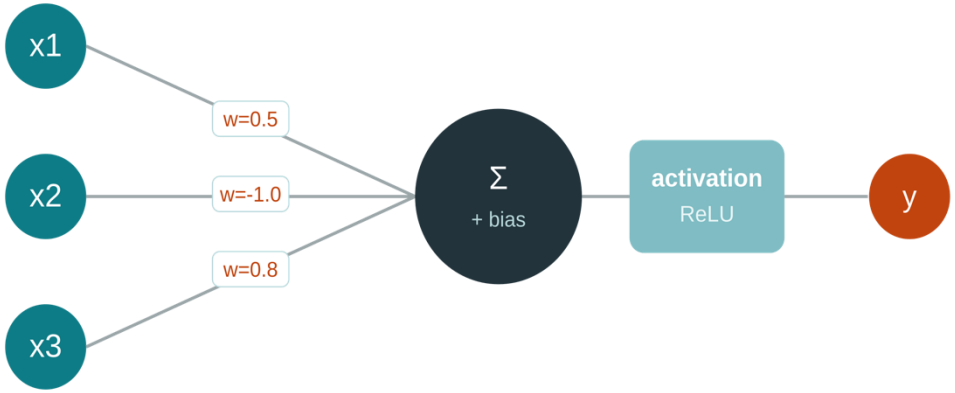
	Regression	Classification
สิ่งที่ทำนาย	ค่าต่อเนื่อง	หมวดหมู่
ตัวอย่าง	ทำนายราคาบ้าน	ภาพนี้เป็นเสื้อหรือรองเท้า
ผลลัพธ์	ตัวเลข เช่น 3,250,000 บาท	คลาส + ความน่าจะเป็น
ตัวชี้วัด	ค่าคลาดเคลื่อนเฉลี่ย	ความถูกต้อง (accuracy)

แล็บช่วงบ่ายเป็น classification ทั้งหมด แต่ควรรู้ว่ามีอีกประเภท

ปูพื้น

การเรียนรู้ของโมเดล คือการปรับตัวเลขชุดหนึ่ง

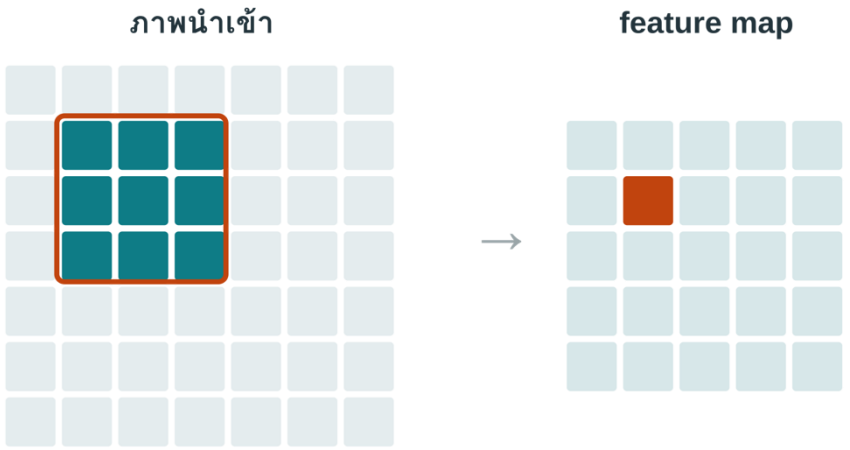
- โมเดลคือฟังก์ชันที่มีตัวเลขภายในจำนวนมาก เรียกว่า parameter
- parameter หลักคือ weight และ bias
- ตอนเริ่มต้น ค่าถูกสุ่ม โมเดลจึงทายมั่ว
- การฝึก คือค่อย ๆ ปรับให้ทายผิดน้อยลง
- เนื้อหาที่เหลือทั้งวัน คือรายละเอียดของประโยคนี้



ปูพื้น

Feature และ Label

- Feature คือข้อมูลนำเข้าที่ใช้ทำนาย
- Label คือคำตอบที่ต้องการให้ทำนาย
- ทำนายราคาบ้าน: feature = ขนาด ทำเล / label = ราคา
- จำแนกภาพ: feature = ค่าพิกเซล / label = ชื่อคลาส
- ทุกอย่างต้องแปลงเป็นตัวเลขก่อนเข้าโมเดลเสมอ



ปูพื้น

การแบ่งข้อมูลออกเป็นสามชุด



หากไม่แบ่ง จะแยกไม่ออกว่าโมเดลเข้าใจจริงหรือเพียงจดจำข้อมูลฝึก — เปรียบเหมือนทำแบบฝึกหัด สอบย่อย และสอบปลายภาค

ส่วนที่ 2

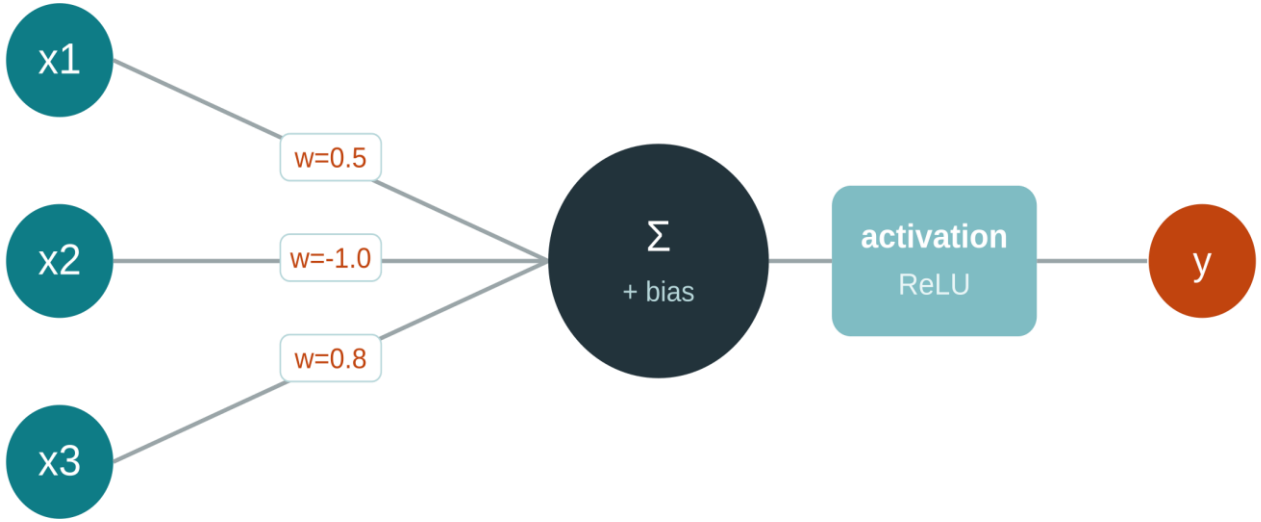
Neural Network ทำงานอย่างไร

เนื้อหาแกนกลางของช่วงเช้า

~60 นาที

Neural Network

Perceptron หน่วยคำนวณที่เล็กที่สุด



องค์ประกอบ

- รับค่านำเข้าหลายค่า แต่ละค่ามี weight
- คูณด้วย weight รวมกัน บวก bias
- ผลรวม (z) ส่งเข้า activation function
- neural network สร้างจากหน่วยแบบนี้จำนวนมาก

Neural Network

ตัวอย่าง: คำนวณ perceptron ด้วยมือ

```
กำหนดให้   x1 = 2,  x2 = 3
weight     w1 = 0.5,  w2 = -1.0,  bias b = 1.0

z  =  (2 x 0.5)  +  (3 x -1.0)  +  1.0
   =  1.0  -  3.0  +  1.0   =   -1.0

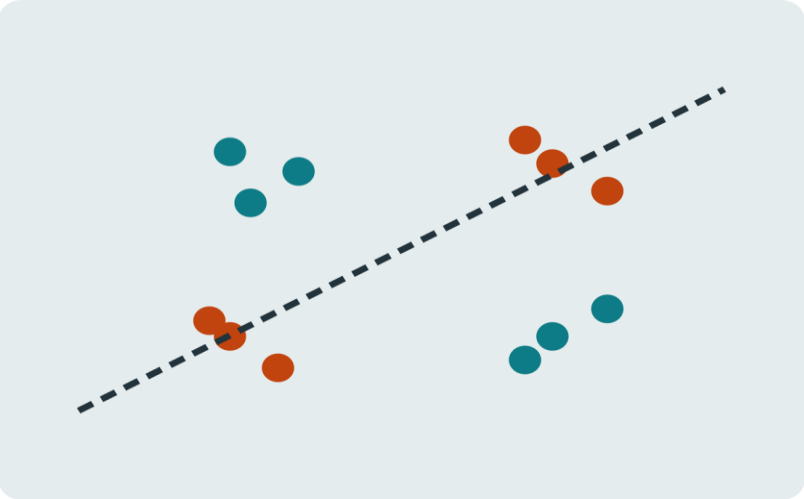
ReLU(z) = max(0, -1.0) = 0   ->  หน่วยนี้ไม่ส่งสัญญาณ
```

ถ้าปรับ w2 จาก -1.0 เป็น -0.2 จะได้ z = 1.4 และผลลัพธ์เปลี่ยนทันที

Neural Network

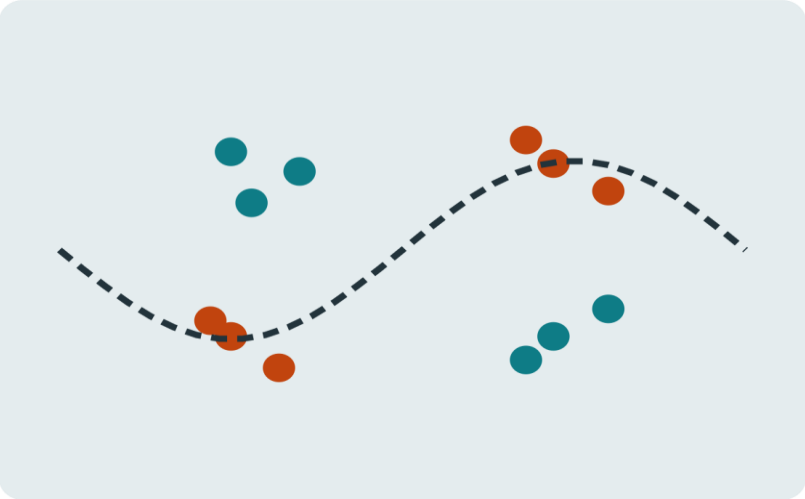
ข้อจำกัดของหน่วยเดียว

ไม่มี activation (เส้นตรง)



เส้นตรงเดียวแบ่งไม่ได้

มี activation (โค้งได้)



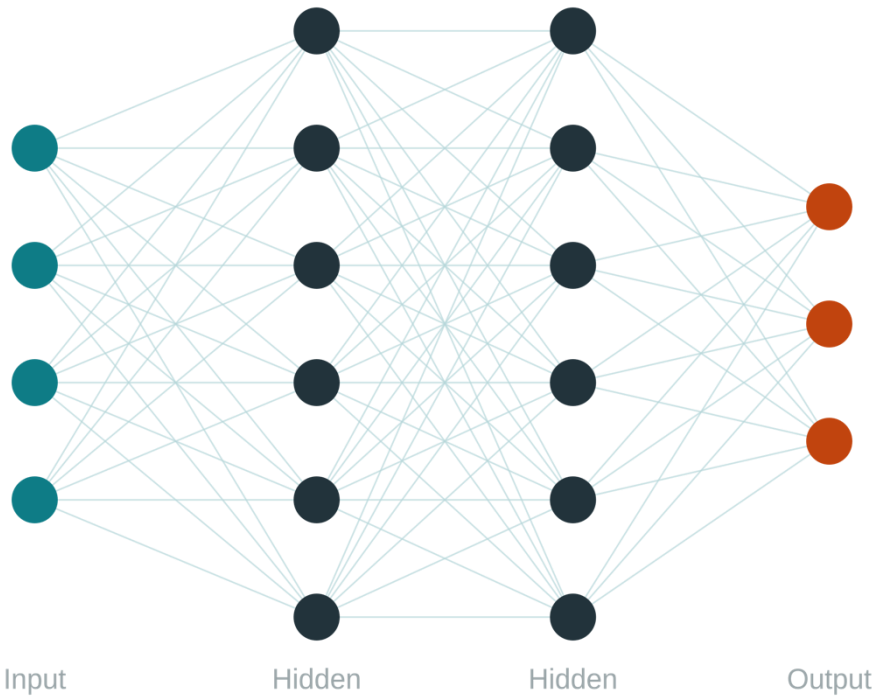
แบ่งได้ถูกต้อง

หน่วยเดียวแบ่งข้อมูลได้ด้วยเส้นตรงเท่านั้น ข้อมูลจริงจำนวนมากแบ่งด้วยเส้นตรงเส้นเดียวไม่ได้ (เช่นปัญหา XOR)

Neural Network

Multilayer Perceptron

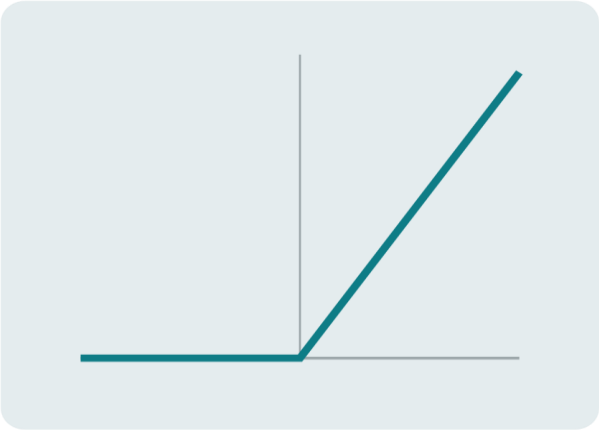
- Input layer รับข้อมูลนำเข้า
- Hidden layer อยู่ตรงกลาง จะมีกี่ชั้นก็ได้
- Output layer ให้ผลลัพธ์สุดท้าย
- "deep" หมายถึงมี hidden layer หลายชั้น
- แต่ละชั้นสกัดลักษณะที่ซับซ้อนขึ้น



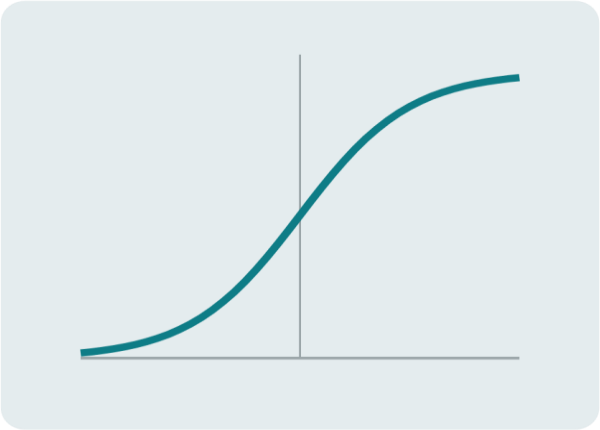
Neural Network

Activation function มีไว้ทำอะไร

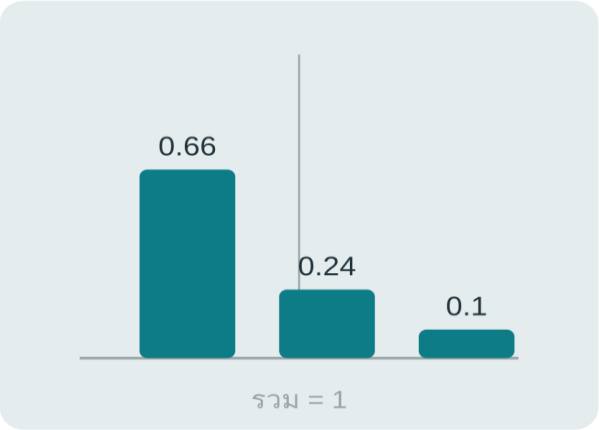
ReLU



Sigmoid



Softmax



เพิ่มความไม่เป็นเชิงเส้นให้โมเดล ทำให้สร้างขอบเขตการตัดสินใจแบบโค้งได้ เลือกใช้ต่างกันตามตำแหน่งชั้นและประเภทโจทย์

Neural Network

ReLU และ Sigmoid

ReLU

- สูตร max(0, z) ค่าติดลบเป็นศูนย์
- ใช้ใน hidden layer เป็นมาตรฐาน
- คำนวณเร็ว ฝึกโมเดลลึกได้ดี
- ระวัง: หน่วยที่ติดลบตลอดหยุดเรียนรู้

Sigmoid

- บีบค่าให้อยู่ระหว่าง 0 ถึง 1
- ใช้ที่ output ของโจทย์สองทาง
- ตีความเป็นความน่าจะเป็นได้
- ไม่ใช้ใน hidden layer (ค่าย้อนกลับจาง)

Neural Network

ตัวอย่าง: Softmax แปลงคะแนนดิบเป็นความน่าจะเป็น

```
คะแนนดิบจาก output layer   ->   [ 2.0 ,  1.0 ,  0.1 ]

หลังผ่าน softmax           ->   [ 0.66 ,  0.24 ,  0.10 ]

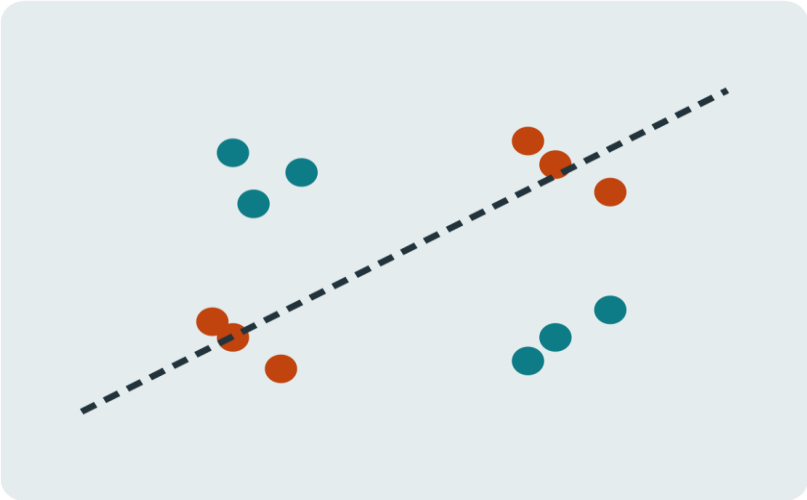
   - ทุกค่าอยู่ระหว่าง 0 ถึง 1
   - ผลรวมทุกคลาส = 1 พอดี
   - คลาสที่คะแนนสูงกว่า ได้ความน่าจะเป็นสูงกว่า
```

โมเดลไม่ตอบว่า "ใช่/ไม่ใช่" แต่ตอบเป็นระดับความมั่นใจเสมอ

Neural Network

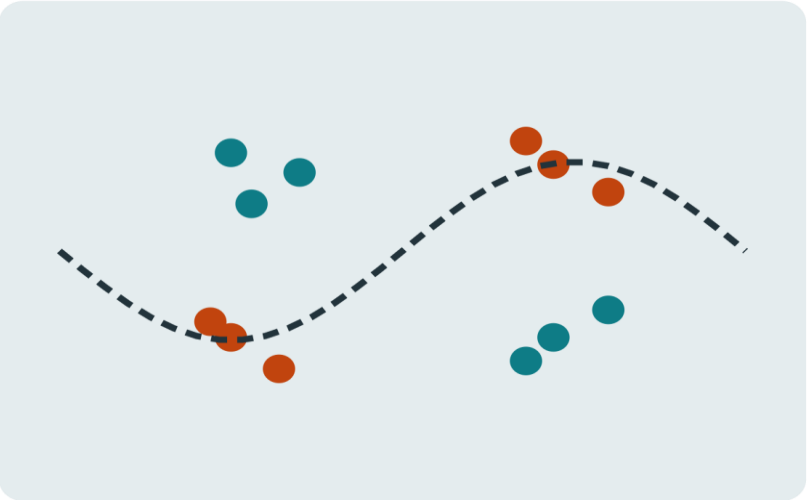
หากไม่มี activation function จะเกิดอะไรขึ้น

ไม่มี activation (เส้นตรง)



เส้นตรงเดียวแบ่งไม่ได้

มี activation (โค้งได้)



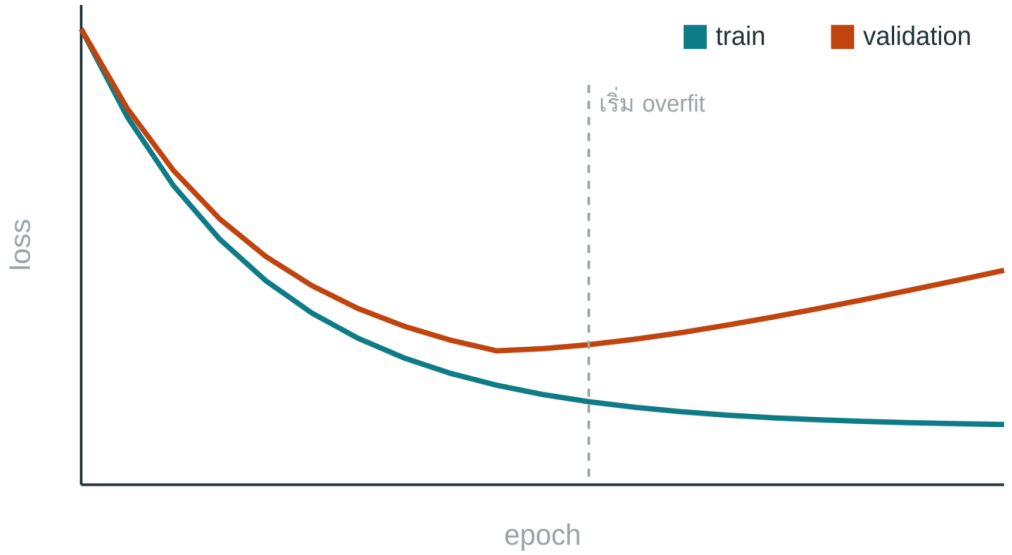
แบ่งได้ถูกต้อง

การแปลงเชิงเส้นซ้อนกันหลายชั้น ยังเทียบเท่าการแปลงเชิงเส้นชั้นเดียว — activation จึงเป็นสิ่งที่ทำให้การเพิ่มชั้นมีความหมาย

Neural Network

Loss function วัดความผิดพลาดเป็นตัวเลข

- เปรียบเทียบคำตอบที่ทายกับคำตอบจริง
- สรุปเป็นตัวเลขเพียงค่าเดียว ยิ่งน้อยยิ่งดี
- ไม่มีตัวเลขนี้ ก็ไม่รู้จะปรับไปทางใด
- ต้องเลือกให้ตรงประเภทโจทย์
- คือค่าที่เห็นเลื่อนบนจอตอนสั่งฝึก



Neural Network

เลือก loss function และ activation ชั้นสุดท้ายอย่างไร

ประเภทโจทย์	Activation ชั้นสุดท้าย	Loss function
ทำนายค่าต่อเนื่อง	ไม่ใส่ (linear)	Mean Squared Error
จำแนกสองคลาส	Sigmoid (1 หน่วย)	Binary Crossentropy
จำแนกหลายคลาส	Softmax (เท่าจำนวนคลาส)	Categorical Crossentropy

สไลด์นี้ให้ผู้เรียนถ่ายเก็บไว้ เป็นจุดที่ผิดกันบ่อยที่สุดตอนเขียนโค้ด

ตัวอย่าง: Crossentropy ลงโทษความมั่นใจที่ผิด

สมมติคำตอบที่ถูกต้องคือคลาส A

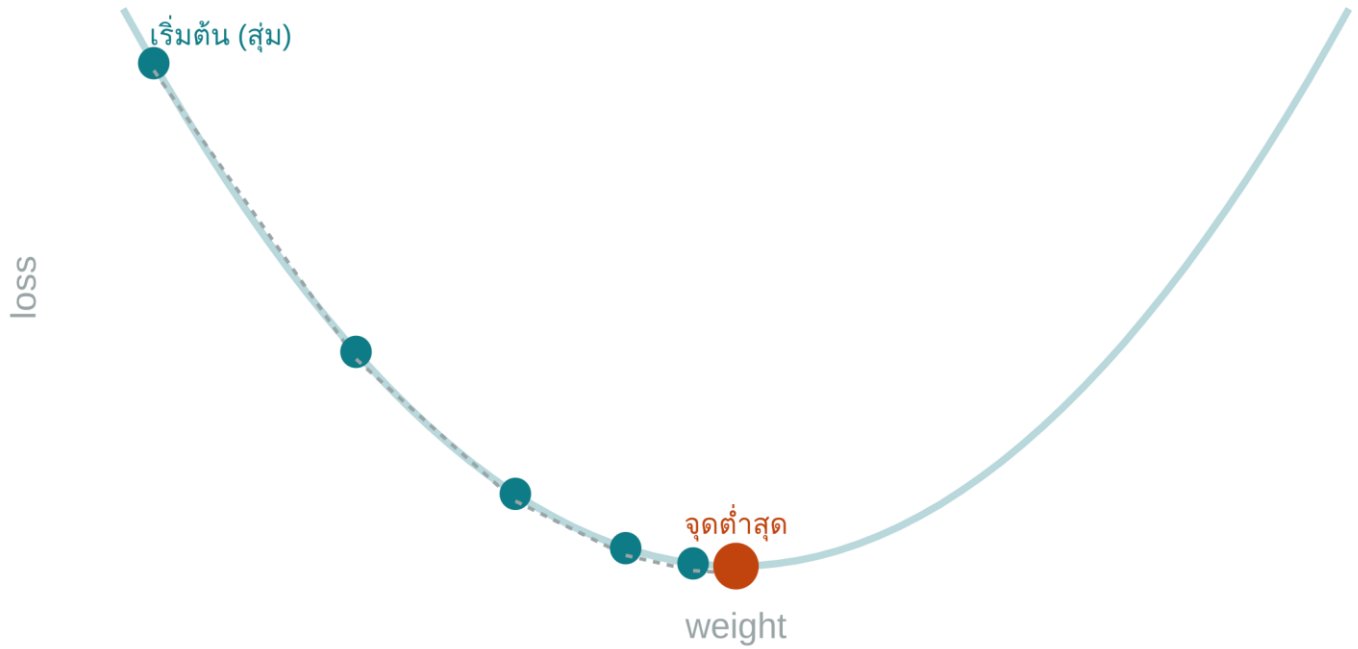
ให้ความน่าจะเป็นคลาส A = 0.90 -> loss = 0.11
ให้ความน่าจะเป็นคลาส A = 0.50 -> loss = 0.69
ให้ความน่าจะเป็นคลาส A = 0.10 -> loss = 2.30

(loss = -ln(ความน่าจะเป็นของคลาสที่ถูกต้อง))

ยิ่งมั่นใจในคำตอบที่ผิด ค่าปรับยิ่งสูงขึ้นอย่างรวดเร็ว

Neural Network

Gradient Descent วิธีค้นหาค่าที่ดีขึ้น

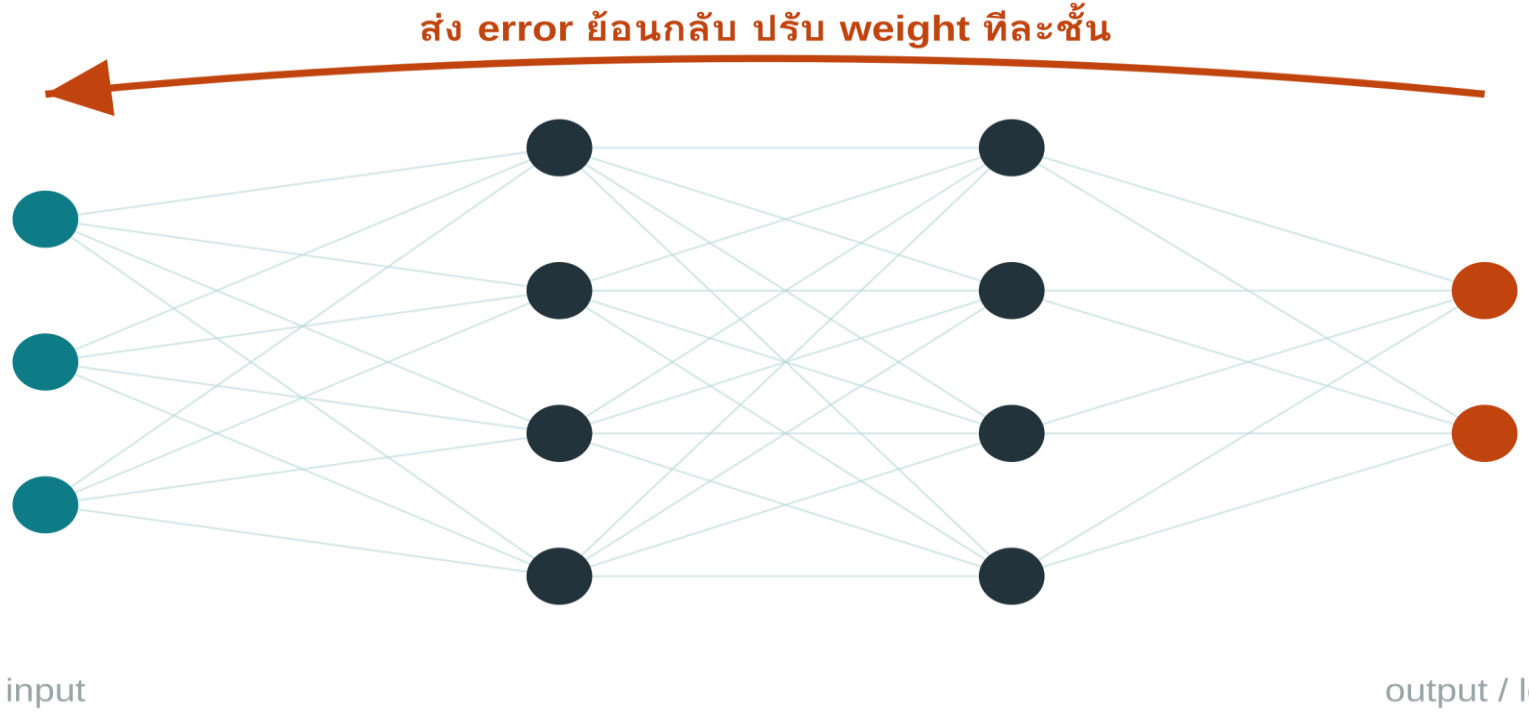


เดินลงเขาในหมอก

- มองไม่เห็นก้นเขา เห็นแค่ความชันใต้เท้า
- ก้าวไปทางที่ลาดลงทีละก้าว
- ความชัน = gradient
- ขนาดก้าว = learning rate

Neural Network

Backpropagation กระจายความรับผิดชอบย้อนกลับ



คำนวณย้อนจากชั้นสุดท้ายกลับไปชั้นแรก แต่ละ weight จึงรู้ว่าควรปรับขึ้นหรือลงเท่าใด (ไม่ต้องคำนวณเอง แต่ควรเข้าใจทิศทาง)

Neural Network

Optimizer ตัวที่ลงมือปรับค่าจริง

SGD

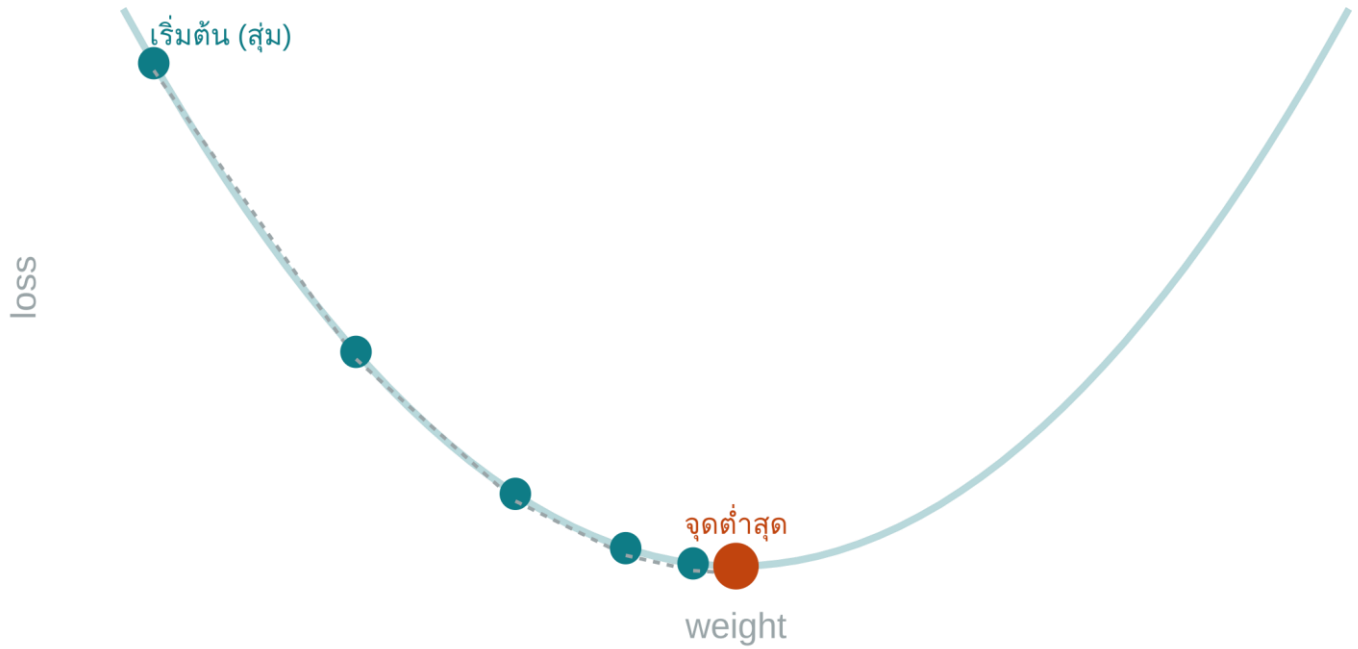
- ปรับตามทิศ gradient ด้วยก้าวคงที่
- เข้าใจง่าย เป็นพื้นฐานของตัวอื่น
- ต้องปรับ learning rate เองระวัง ๆ

Adam

- ปรับก้าวให้เหมาะกับ parameter แต่ละตัว
- มักได้ผลดีตั้งแต่ค่าเริ่มต้น
- ใช้เป็นค่าตั้งต้นในทุกแล็บช่วงบ่าย

Neural Network

Learning Rate ขนาดก้าวที่เหมาะสม



ก้าวใหญ่ไปหรือเล็กไป

- สูงไป — ก้าวข้ามจุดต่ำสุด ค่าแกว่ง
- ต่ำไป — ช้ามาก หรือค้างกลางทาง
- ค่าที่ใช้บ่อยราว 0.001–0.01
- บ่ายมีเทคนิคไล่หาค่าที่ดีที่สุด

Neural Network

ตัวอย่าง: Epoch, Batch และ Step ต่างกันอย่างไร

```
ข้อมูลฝึกทั้งหมด 60,000 ภาพ    batch size = 32

1 step   =  ฝึกด้วย 32 ภาพ แล้วปรับ parameter หนึ่งครั้ง
1 epoch  =  60,000 / 32  =  1,875 steps  (ครบหนึ่งรอบ)
ฝึก 10 epochs  =  ปรับ parameter 18,750 ครั้ง

batch ใหญ่ขึ้น  ->  เร็วขึ้นต่อ epoch แต่กินหน่วยความจำมากขึ้น
```

ทั้งสามคำนี้จะปรากฏในโค้ดและผลลัพธ์บนหน้าจอตลอดช่วงบ่าย

Neural Network

สามขั้นตอนการสร้างโมเดลด้วย Keras

```
model = keras.Sequential([
    layers.Dense(128,
        activation='relu'),
    layers.Dense(10,
        activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical
          _crossentropy',
    metrics=['accuracy'])

model.fit(x_train, y_train,
          epochs=10)
```

- ขั้นที่ 1 กำหนดว่ามีชั้นอะไรบ้าง
- ขั้นที่ 2 กำหนด optimizer, loss, metric
- ขั้นที่ 3 สั่งฝึกด้วยข้อมูลจริง
- ทุกแล็บบ่ายวนอยู่กับสามขั้นนี้

ส่วนที่ 3

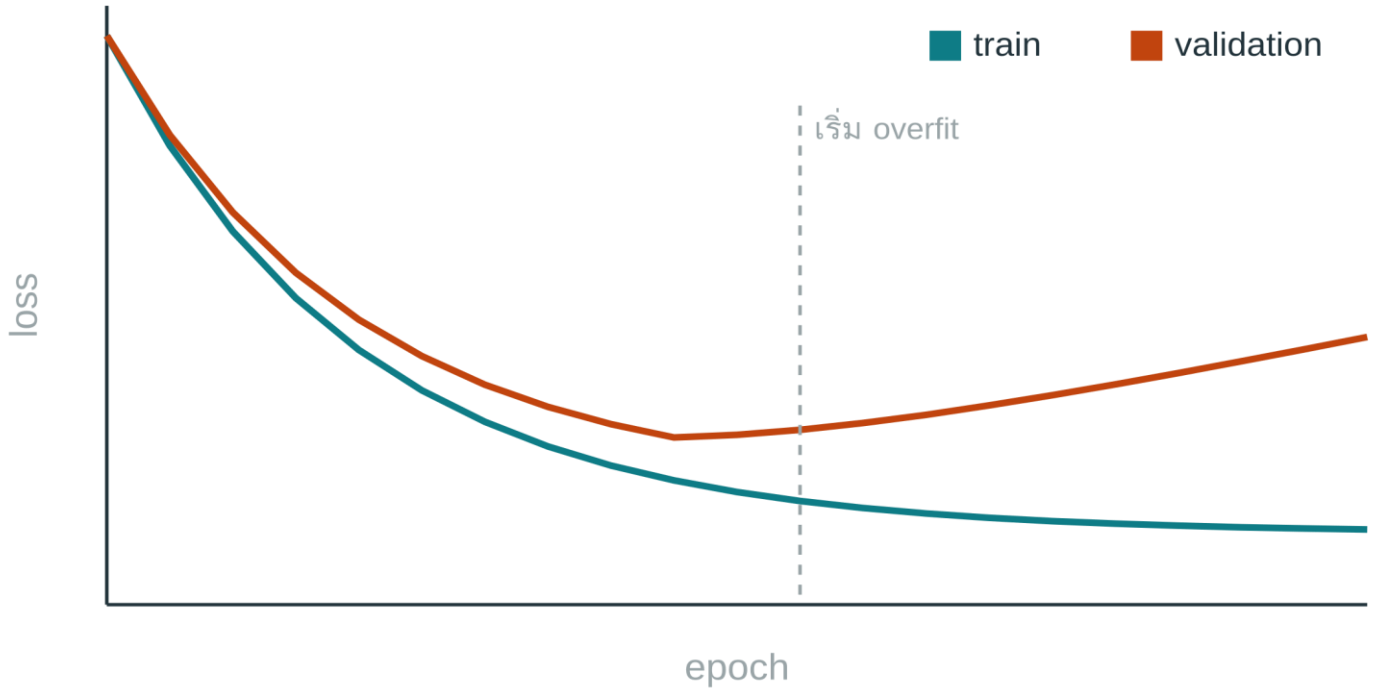
ประเมินผลโมเดล

อ่านกราฟให้เป็นก่อนลงมือปฏิบัติ

~25 นาที

การประเมินผล

การอ่านกราฟ Loss Curve



อ่านกราฟให้เป็น

- แกนนอน = epoch แกนตั้ง = loss
- สองเส้น: train กับ validation
- ลงพร้อมกัน = เรียนรู้ได้ดี
- validation กลับสูง = เริ่มจดจำข้อมูลฝึก
- จุดต่ำสุดของ validation = จุดที่ควรหยุด

การประเมินผล

Underfitting และ Overfitting

Underfitting

- โมเดลเรียบง่ายเกินกว่าจะจับรูปแบบ
- loss สูงทั้ง training และ validation
- แก้: เพิ่มความซับซ้อน หรือฝึกนานขึ้น

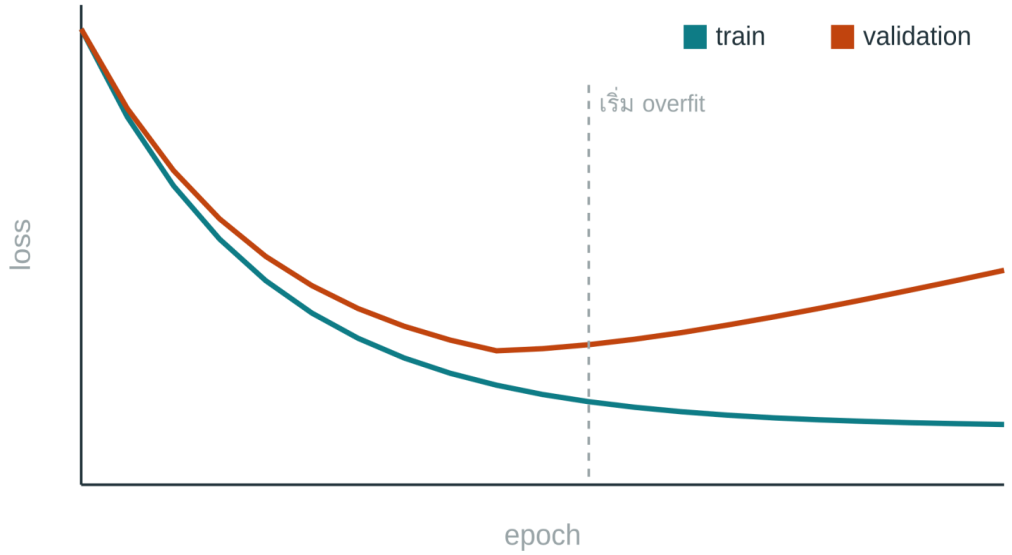
Overfitting

- โมเดลจดจำรายละเอียดเฉพาะข้อมูลฝึก
- training loss ต่ำ แต่ validation สูง
- แก้: ดูรายละเอียดในสไลด์ถัดไป

การประเมินผล

แนวทางรับมือ Overfitting

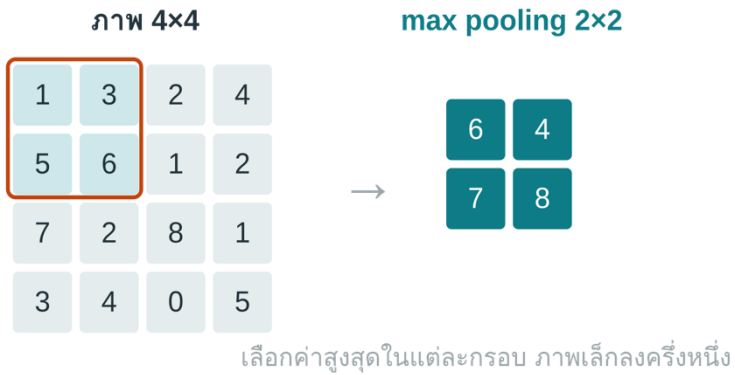
- เพิ่มข้อมูล — ได้ผลดีที่สุด แต่แพงที่สุด
- Data augmentation — พลิก หมุน ซูม ครอบตัด
- Dropout — สุ่มปิดบางหน่วยระหว่างฝึก
- Early stopping — หยุดเมื่อ validation ไม่ดีขึ้น
- ลดขนาดโมเดล ในกรณีโมเดลใหญ่เกินข้อมูล



การประเมินผล

ค่า Accuracy เพียงอย่างเดียวไม่เพียงพอ

- ตัวเลขเดียวซ่อนรายละเอียดไว้มาก
- Confusion matrix แสดงว่าสับสนคลาสใดกับคลาสใด
- เช่น เสื้อเชิ้ต เสื้อกันหนาว เสื้อโค้ท มักสับสนกัน
- ข้อมูลนี้ชี้ทางว่าควรแก้ที่ข้อมูลหรือที่โมเดล
- ช่วงบ่ายจะได้สร้าง confusion matrix ของตนเอง



เลือกค่าสูงสุดในแต่ละกรอบ ภาพเล็กลงครึ่งหนึ่ง

ส่วนที่ 4

CNN สำหรับข้อมูลภาพ

เหตุใดภาพจึงต้องใช้สถาปัตยกรรมเฉพาะ

~45 นาที



CNN

เหตุใด Dense layer จึงไม่เหมาะกับภาพ

```
ภาพ 224 x 224 พิกเซล 3 ช่องสี  =  150,528 ค่านำเข้า

ต่อเข้า Dense 128 หน่วย
   weight = 150,528 x 128 = 19.3 ล้านตัว   (เฉพาะชั้นแรก)

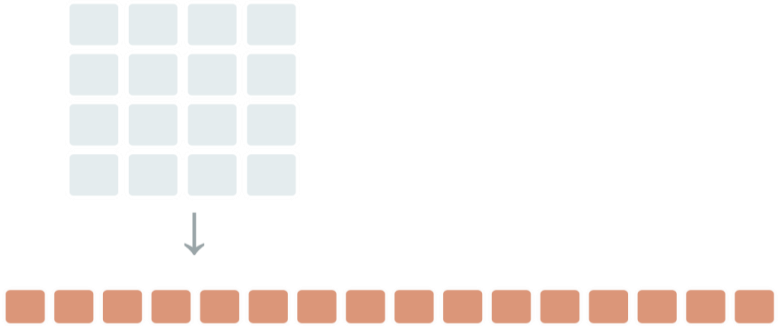
ใช้ Conv filter 3 x 3 จำนวน 32 ตัว
   weight = 3 x 3 x 3 x 32 = 864 ตัว
```

ต่างกันกว่าสองหมื่นเท่า และ CNN ยังให้ผลดีกว่าด้วย

CNN

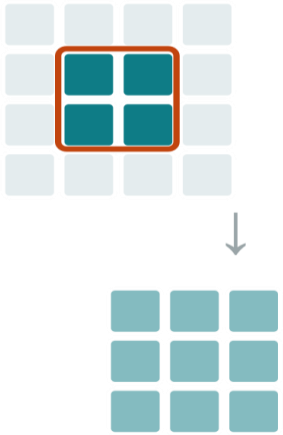
ปัญหาที่สองของ Dense layer กับภาพ

MLP: คลี่ภาพทิ้งตำแหน่ง



พิกเซลกลายเป็นแถวเดียว ความสัมพันธ์หาย

CNN: รักษาโครงสร้าง 2 มิติ

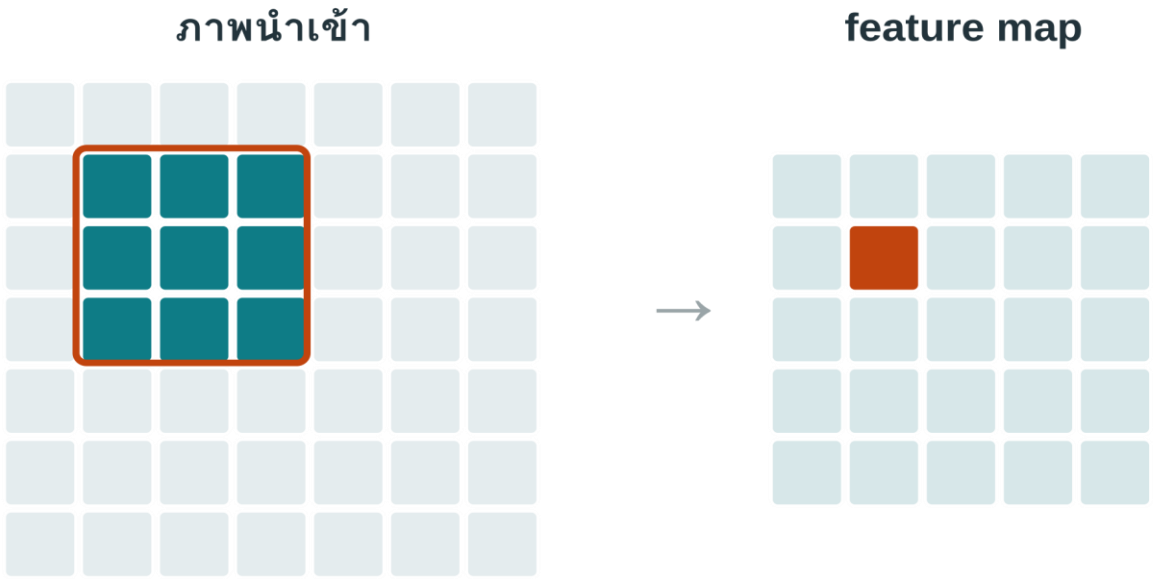


filter เลื่อนทั่วภาพ เก็บความสัมพันธ์ไว้

Dense มองทุกพิกเซลเป็นค่าอิสระ หากวัตถุขยับเล็กน้อย ค่านำเข้าเปลี่ยนหมด CNN ถูกออกแบบมาแก้ปัญหานี้โดยตรง

CNN

Convolution ทำงานอย่างไร



หัวใจของ CNN

- ใช้ kernel เล็ก ๆ เลื่อนไปทับภาพ
- แต่ละตำแหน่ง คูณแล้วรวมเป็นค่าหนึ่ง
- ค่าทั้งหมดประกอบเป็น feature map
- โมเดลเรียนค่าใน kernel เอง

ตัวอย่าง: filter ตรวจจับขอบแนวตั้ง

```
kernel  ที่ใช้          บริเวณที่มีขอบ          บริเวณสีเรียบ
  1    0   -1            10   10    0            10   10   10
  1    0   -1            10   10    0            10   10   10
  1    0   -1            10   10    0            10   10   10

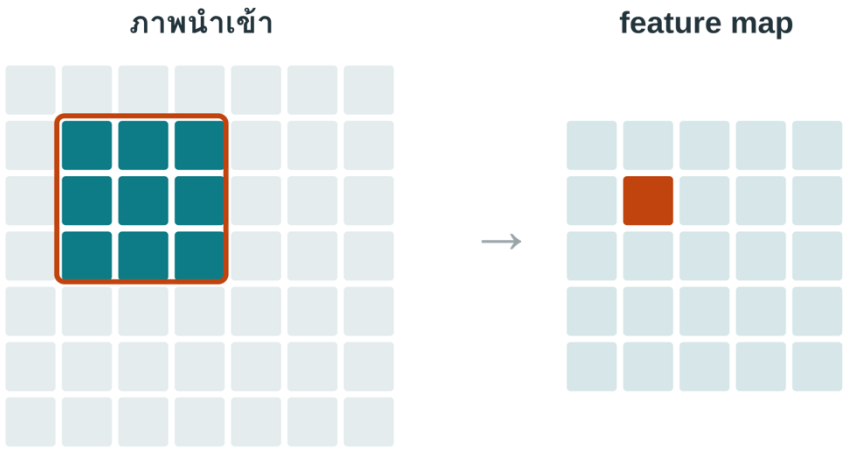
ผลการคำนวณ          ->   30                    ->   0
```

filter หนึ่งตัวตรวจจับลักษณะหนึ่งอย่าง และทำแบบเดียวกันทั่วทั้งภาพ

CNN

หนึ่งชั้นมี filter หลายตัว

- Conv layer หนึ่งชั้นมักมี filter 32–256 ตัว
- แต่ละตัวเรียนตรวจจับลักษณะคนละอย่าง
- ตัวหนึ่งจับขอบตั้ง อีกตัวจับขอบนอน
- โมเดลเรียนเองว่าควรจับอะไร ไม่ได้ตั้งให้
- ผลลัพธ์คือ feature map เท่าจำนวน filter



CNN

ตัวอย่าง: Padding และ Stride ส่งผลต่อขนาดอย่างไร

```
ภาพนำเข้า 28 x 28    kernel 3 x 3

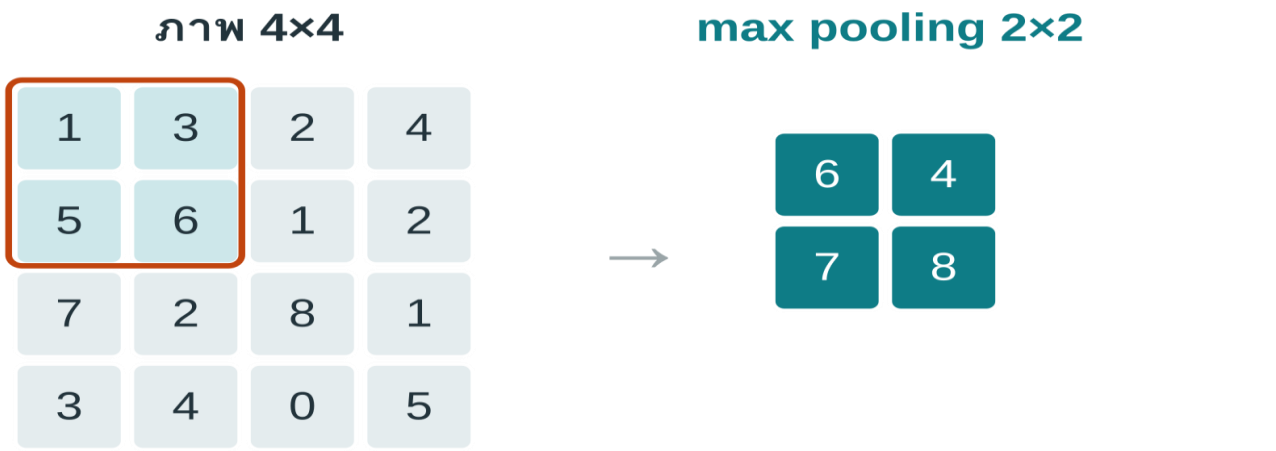
padding='valid'  stride=1   ->   26 x 26   (ขนาดหด)
padding='same'   stride=1   ->   28 x 28   (เติมขอบไว้)
padding='same'   stride=2   ->   14 x 14   (เลื่อนทีละสอง)

ซ้อน conv โดยไม่ padding ภาพจะหดจนหมดก่อนถึงชั้นท้าย
```

padding ควบคุมขอบภาพ ส่วน stride ควบคุมความถี่ในการเลื่อน

CNN

Pooling ลดขนาดแต่คงสาระสำคัญ



เลือกค่าสูงสุดในแต่ละกรอบ ภาพเล็กลงครึ่งหนึ่ง

max pooling 2×2 เลือกค่าสูงสุดในทุกกรอบ ภาพเล็กลงครึ่งหนึ่ง ทนต่อการขยับตำแหน่งเล็กน้อย และไม่มี parameter ให้เรียนรู้

CNN

Feature Hierarchy สิ่งที่แต่ละชั้นมองเห็น



ชั้นต้นจับขอบและเส้น ชั้นกลางจับลวดลาย ชั้นลึกจับรูปทรงที่มีความหมาย — นี่คือเหตุผลที่ยิ่งลึกยิ่งเก่งขึ้น และที่ transfer learning ใช้ได้ผล

CNN

โครงสร้างมาตรฐานของ CNN

```
model = keras.Sequential([
  layers.Conv2D(32, 3,
     activation='relu'),
  layers.MaxPooling2D(),

  layers.Conv2D(64, 3,
     activation='relu'),
  layers.MaxPooling2D(),

  layers.Flatten(),
  layers.Dense(10,
     activation='softmax')
])
```

- ทำซ้ำเป็นคู่ Conv แล้วตามด้วย Pool
- จำนวน filter เพิ่มขึ้นเมื่อชั้นลึกขึ้น
- Flatten แปลงเป็นแถวเดียวก่อนเข้า Dense
- ปิดท้ายด้วย Dense เท่าจำนวนคลาส

CNN

พัฒนาการของสถาปัตยกรรมที่ควรรู้จัก

สถาปัตยกรรม	ปี	จุดเด่นที่นำเสนอ
LeNet-5	1998	พิสูจน์ว่า convolution ใช้ได้จริงกับการอ่านตัวเลข
AlexNet	2012	ใช้ GPU ฝึกโมเดลลึก จุดเริ่มกระแส deep learning
VGG	2014	โครงเรียบง่าย ใช้ kernel 3×3 ซ้อนจำนวนมาก
ResNet	2015	เพิ่มทางลัดข้ามชั้น ฝึกลึกนับร้อยชั้นได้

ผ่านอย่างรวดเร็วเพื่อให้เห็นบริบท ไม่ต้องจำรายละเอียด

ส่วนที่ 5

Transfer Learning

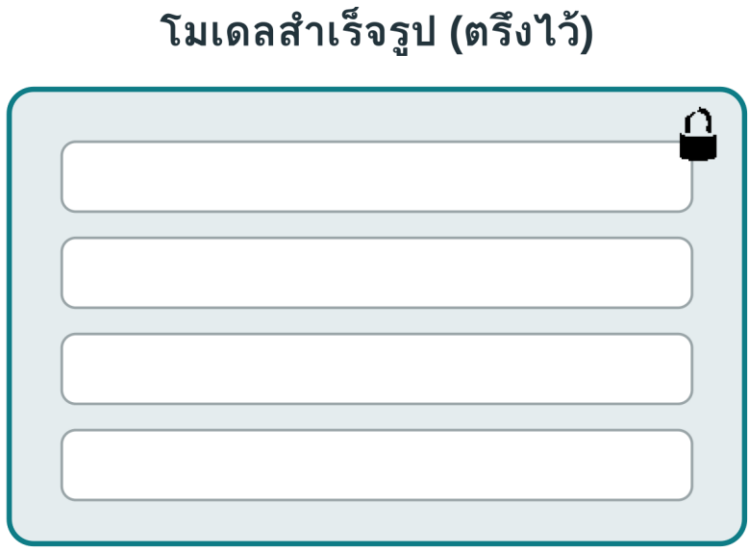
ใช้ประโยชน์จากโมเดลที่ผู้อื่นฝึกไว้แล้ว

~30 นาที



Transfer Learning

เหตุใดจึงไม่ต้องเริ่มฝึกจากศูนย์



→ **ชั้นของเรา** (ฝึกใหม่)

ยืมความรู้มาใช้

- โมเดลเผยแพร่ผ่านการฝึกด้วยภาพหลายล้านใบ
- ชั้นต้นเรียนขอบ เส้น ลวดลายไว้แล้ว
- ความรู้ระดับนี้ใช้ได้กับงานภาพแทบทุกแบบ
- เราแค่เปลี่ยนชั้นท้ายให้ตรงงานของเรา

Transfer Learning

ImageNet ต้นทางของโมเดลสำเร็จรูป

- ชุดข้อมูลภาพขนาดใหญ่ที่เป็นมาตรฐานของวงการ
- มีภาพราว 1.2 ล้านภาพ ครอบคลุม 1,000 หมวดหมู่
- โมเดลใน keras.applications ล้วนฝึกด้วยชุดนี้
- แม้หมวดหมู่ไม่ตรงงานเรา ลักษณะพื้นฐานยังใช้ได้
- การฝึกใหม่ทั้งหมดใช้เวลาและค่าใช้จ่ายสูงมาก



Transfer Learning

สองวิธีใช้งานโมเดลสำเร็จรูป

Feature Extraction

- ตรึง parameter เดิมไว้ทั้งหมด
- เปลี่ยนเฉพาะชั้นท้ายให้ตรงคลาสเรา
- ฝึกเฉพาะชั้นที่เพิ่ม จึงเร็วมาก
- เหมาะเมื่อข้อมูลน้อย งานใกล้เคียงเดิม

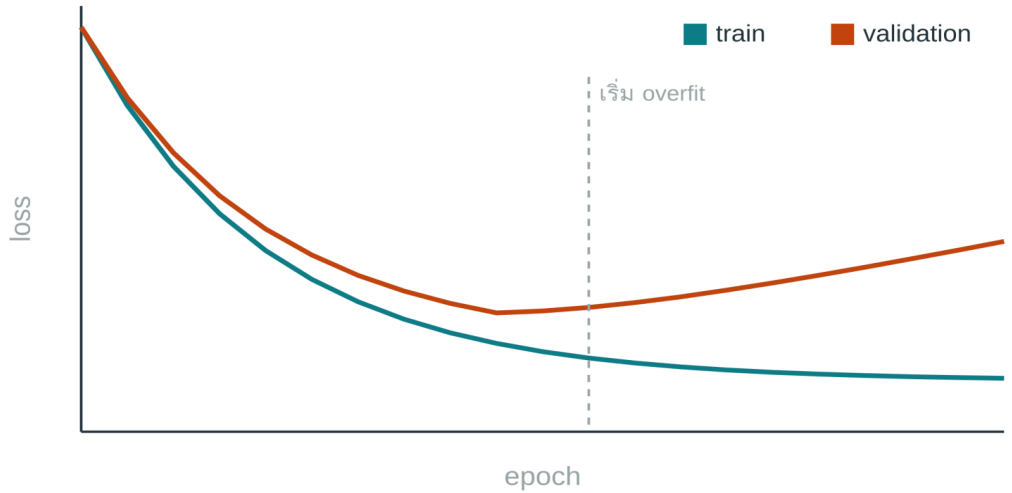
Fine-tuning

- ปลดล็อกชั้นท้ายให้ปรับตามข้อมูลใหม่
- ใช้ learning rate ต่ำกว่าปกติ
- ให้ผลดีกว่า แต่ใช้เวลา/ข้อมูลมากกว่า
- ทำหลัง feature extraction ได้ผลแล้วเสมอ

Transfer Learning

ผลลัพธ์ที่จะได้เห็นในช่วงบ่าย

- แล็บใช้ข้อมูลเพียงร้อยละ 10 ของชุดเต็ม
- กรณีที่หนึ่ง ฝึก CNN ขึ้นใหม่ทั้งหมด
- กรณีที่สอง ใช้ transfer learning ชุดเดียวกัน
- เปรียบเทียบทั้งความถูกต้องและเวลาที่ใช้ฝึก
- ตัวเลขจริงจะเติมหลังทดลองรันแล้ว



Transfer Learning

โมเดลสำเร็จรูปจาก keras.applications

โมเดล	ขนาด	เหมาะกับ
MobileNetV2	เล็ก	เครื่องทรัพยากรจำกัด งานที่ต้องการความเร็ว
EfficientNetV2	กลาง	สมดุลความถูกต้องกับขนาด ใช้เป็นตัวเลือกแรก
ResNet50	ใหญ่	งานที่ต้องการความถูกต้องสูง มีทรัพยากรพอ

แล็บช่วงบ่ายจะใช้ EfficientNetV2 เป็นหลัก

ส่วนที่ 6

สะพานสู่เนื้อหาวันถัดไป

จากข้อมูลภาพ สู่ข้อมูลภาษา

~10 นาที



สะพานสู่ Day 2

ภาพกับข้อความมีโครงสร้างต่างกัน

ข้อมูลภาพ

- โครงสร้างเชิงพื้นที่ สองมิติ
- พิกเซลที่อยู่ใกล้กันสัมพันธ์กัน
- ขนาดข้อมูลคงที่แน่นอน
- CNN ใช้ประโยชน์จากความใกล้นี้

ข้อมูลข้อความ

- โครงสร้างเชิงลำดับ มิติเดียว
- คำที่ห่างกันมากอาจสัมพันธ์กันมาก
- ความยาวไม่คงที่
- จึงต้องใช้สถาปัตยกรรมคนละแบบ

RNN และข้อจำกัดที่นำไปสู่ Transformer



RNN อ่านทีละคำและส่งความจำต่อไป แต่ลำดับยาวทำให้ข้อมูลต้นทางจางหาย และต้องประมวลผลตามลำดับ จึงขนานการคำนวณไม่ได้



หากให้โมเดลมองทุกคำในประโยคพร้อมกัน แล้วเลือกเองว่าจะให้น้ำหนักกับคำใด จะเป็นอย่างไร

Day 2 · Attention และสถาปัตยกรรม Transformer